Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЛАБОРАТОРНАЯ РАБОТА № 3-4

«Функциональные возможности языка Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции конструкции языков программирования»

Выполнил: студент гр. ИУ5-33Б _____ (Ахметзянова Г.Т.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Оргсян О.Г.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель: изучение возможностей функционального программирования в языке Python.

Задание: решить несколько задач с использованием возможностей функционального программирования в языке Python, расположив файлы с решениями отдельных задач в пакете lab_python_fp с выводом тестовых результатов при запуске каждого файла.

Выполнение работы.

1. Разработка генератора для последовательной выборки значений одного или нескольких полей из списка словарей с игнорированием пустых значений представлена в листинге 1:

Листинг 1 – Функция-генератор field() из файла field.py

```
def field(items, *args):
    assert len(args) > 0

    if len(args) == 1:
        key = args[0]
        for item in items:
            if key in item and item[key] is not None:
                yield item[key]
    else:
        for item in items:
            result_dict = {}
            for key in args:
                if key in item and item[key] is not None:
                    result_dict[key] = item[key]
            if len(result_dict) > 0:
                yield result_dict
```

2. Примеры выполнения генератора выборки полей из списка словарей изображены на рисунке 1:

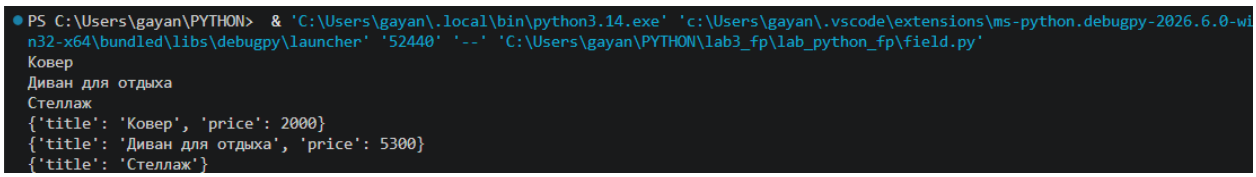


Рисунок 1 – Примеры работы программы из файла field.py

3. Разработка генератора для получения заданного количества случайных целых чисел в указанном диапазоне представлена в листинге 2:

Листинг 2 – Функция-генератор gen_random() из файла gen_random.py

```
def gen_random(num_count, begin, end):
    for _ in range(num_count):
        yield random.randint(begin, end)
```

4. Примеры работы генератора случайных целых чисел изображены на рисунке 2:

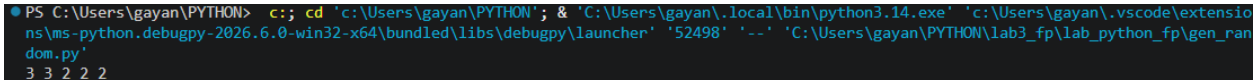


Рисунок 2 – Примеры работы программы из файла gen_random.py

5. Реализация класса-итератора для фильтрации дубликатов элементов последовательности с поддержкой параметра игнорирования регистра строк представлена в листинге 3:

Листинг 3 – Класс-итератор Unique из файла unique.py

```
class Unique(object):
    def __init__(self, items, **kwargs):
        self.ignore_case = kwargs.get('ignore_case', False)
        self.items = iter(items)
        self.seen = set()

    def __next__(self):
        while True:
            item = next(self.items)
            check_item = item
            if self.ignore_case and isinstance(item, str):
                check_item = item.lower()

            if check_item not in self.seen:
                self.seen.add(check_item)
                return item

    def __iter__(self):
        return self
```

6. Примеры работы итератора уникальных значений изображены на рисунке 3:

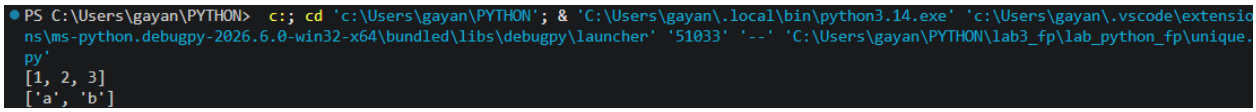


Рисунок 3 – Примеры работы программы из файла unique.py

7. Сортировка списка чисел по убыванию абсолютного значения двумя способами (с помощью встроенной функции и с использованием анонимной lambda-функции) представлена в листинге 4:

Листинг 4 – Сортировка списка по модулю из файла sort.py

```
data = [4, -30, 100, -100, 123, 1, 0, -1, -4]

if __name__ == '__main__':
    result = sorted(data, key=abs, reverse=True)
    print(result)

    result_with_lambda = sorted(data, key=lambda x: abs(x),
reverse=True)
    print(result_with_lambda)
```

8. Результаты сортировки массива чисел по модулю изображены на рисунке 4:

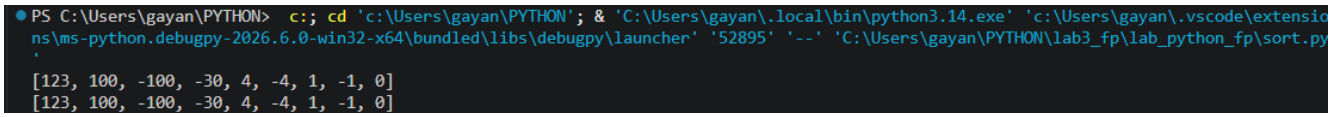


Рисунок 4 – Примеры работы программы из файла sort.py

9. Разработка декоратора для вывода имени вызываемой функции и форматированного результата ее выполнения в зависимости от типа возвращаемых данных представлена в листинге 5:

Листинг 5 – Декоратор print_result() из файла print_result.py

```
def print_result(func):
    def wrapper(*args, **kwargs):
        print(func.__name__)
        res = func(*args, **kwargs)
        if isinstance(res, list):
            for item in res:
                print(item)
        elif isinstance(res, dict):
            for key, val in res.items():
                print(f"{key} = {val}")
        else:
            print(res)
        return res

    return wrapper
```

10. Примеры вызова тестовых функций с декоратором изображены на рисунке 5:

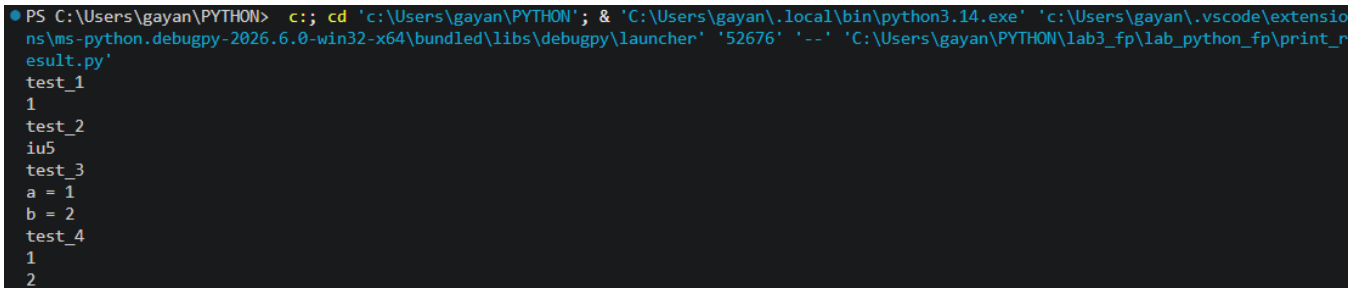


Рисунок 5 – Примеры работы программы из файла print_result.py

11. Реализация контекстных менеджеров для подсчета и вывода времени выполнения блока кода на основе класса и с помощью библиотеки contextlib представлена в листинге 6:

Листинг 6 – Контекстные менеджеры cm_timer_1 и cm_timer_2 из файла cm_timer.py

```
class cm_timer_1:
    def __enter__(self):
        self.start_time = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        end_time = time.time()
        print(f"time: {round(end_time - self.start_time,
2)}")


@contextmanager
def cm_timer_2():
    start_time = time.time()
    try:
        yield
    finally:
        end_time = time.time()
        print(f"time: {round(end_time - start_time, 2)}")
```

12. Примеры измерения времени работы контекстных менеджеров изображены на рисунке 6:

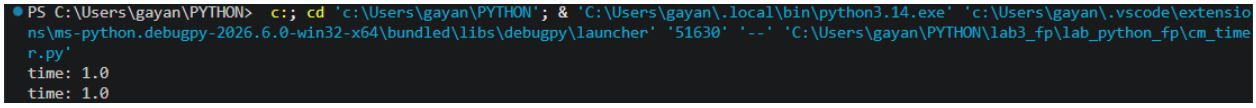


Рисунок 6 – Примеры работы программы из файла cm_timer.py

13. Совместное использование разработанных средств функционального программирования для последовательной фильтрации, модификации и генерации зарплат для списка вакансий из файла JSON представлено в листинге 7:

Листинг 7 – Функции обработки данных f1, f2, f3, f4 из файла process_data.py

```
@print_result
def f1(arg):
    return sorted(Unique(field(arg, 'job-name'),
ignore_case=True), key=str.lower)

@print_result
def f2(arg):
    return list(filter(lambda x:
x.lower().startswith('программист'), arg))

@print_result
def f3(arg):
    return list(map(lambda x: f"{x} с опытом Python", arg))

@print_result
def f4(arg):
    salaries = gen_random(len(arg), 100000, 200000)
    return [f"{job}, зарплата {salary} руб." for job,
salary in zip(arg, salaries)]

if __name__ == '__main__':
    with cm_timer_1():
        f4(f3(f2(f1(data))))
```

14. Результаты конвейерной обработки данных о вакансиях изображены на рисунке 7:

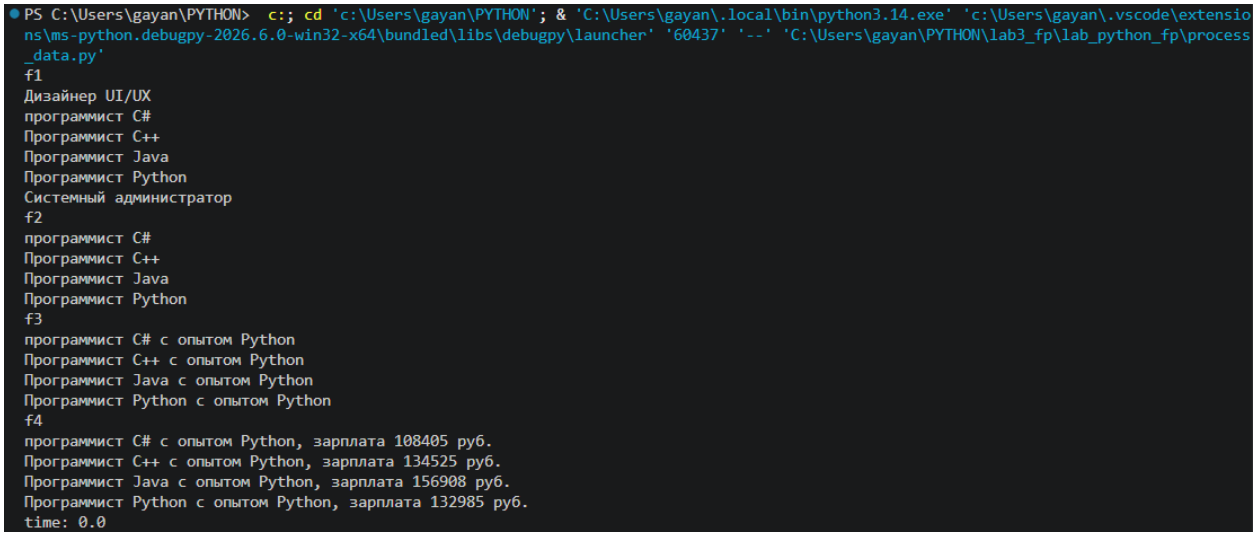


Рисунок 7 – Примеры работы программы из файла process_data.py

Вывод: в результате выполнения лабораторной работы были изучены возможности функционального программирования в языке Python и разработаны программы с применением генераторов, итераторов, анонимных lambda-функций, декораторов и контекстных менеджеров.